

RAUL LAB · DOCUMENTACIÓN CANÓNICA DERIVADA DE MKDOCS

Replicant Lab

Infraestructura, hosts, red, despliegues y operación.

Versión reproducible

sha256:270eee6812acb60001a9093bada9188ed98f92831ba6147df7608caa116004f7

Páginas fuente

28

Diagramas Mermaid

5

Índice

[Inicio](#)

[Arquitectura](#)

FASES

[01 · Concepto](#)

[02 · Replicant](#)

[03 · Nexus](#)

[04 · Docker y Git](#)

[05 · PoC Salones AV](#)

HOSTS

[Replicant](#)

[Nexus](#)

[DigitalOcean](#)

[GitHub](#)

RED

[Visión general](#)

[Inventario provisional](#)

DESPLIEGUE

[Git](#)

[Docker](#)

OPERACIÓN

[SSH](#)

[Comandos útiles](#)

[Mantenimiento](#)

[Descargas](#)

[Pendientes](#)

APLICACIONES

[Índice](#)

[Salones AV](#)

[Reserva-Pistas-UTP](#)

[Cartera Estratégica](#)

[Autodocumentación](#)

[Decisiones](#)

CHANGE LOG

[9 de agosto de 2026](#)

[8 de agosto de 2026](#)

Replicant Lab

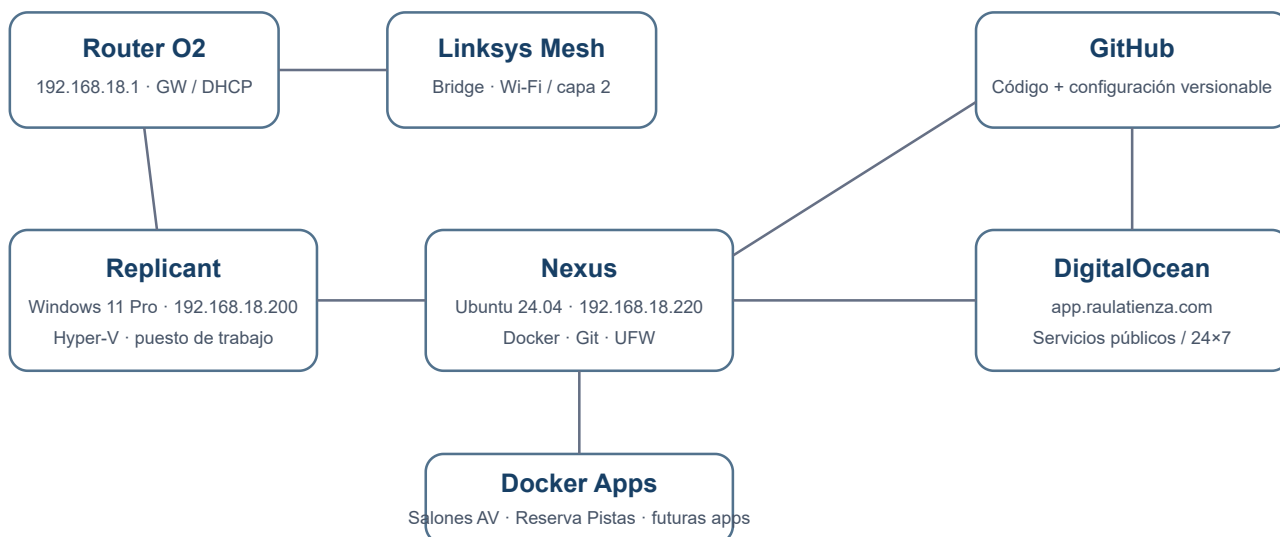
Documentación viva de la infraestructura local y cloud del laboratorio.

Objetivo

Mantener una visión única, entendible y versionada de **concepto, hosts, red, seguridad, Git, Docker y operación**. La documentación funcional de cada aplicación vive en su propio repositorio.

Las versiones portables se descargan desde el icono **↓ Descargas** de la cabecera, junto al selector claro/oscuro.

Arquitectura de un vistazo



La portada usa **SVG nativo**, no Mermaid. Así evitamos que una incompatibilidad del parser pueda romper la vista principal.

Principios

- **Minimalismo:** pocas piezas y cada una con una función clara.
- **Git como fuente de verdad:** código y configuración versionable viven en GitHub.
- **Separación:** Windows para trabajo interactivo; Ubuntu para servicios; cloud para disponibilidad pública/24x7.
- **Persistencia fuera de Git:** datos, secretos y backups no se mezclan con repositorios.
- **Seguridad práctica:** SSH por clave, UFW y puertos publicados de forma explícita.
- **Reproducibilidad:** un host debe poder reconstruirse sin depender de cambios manuales no documentados.

Estado actual

Componente	Estado
Replicant	✅ Operativo

Componente	Estado
Nexus	✅ Operativo
SSH por clave	✅ Operativo
UFW	✅ Activo
Docker / Compose	✅ Operativo
GitHub desde Nexus	✅ Operativo
Salones AV	✅ Observada en Nexus · 8081
Replicant Lab en Nexus	✅ Runtime Nginx estático validado · 8082
Reserva-Pistas-UTP	✅ Validada y observada en Nexus · 8083
Cartera Estratégica	✅ MVP local en Replicant · no desplegada en Nexus
Reserva-Pistas histórico en Nexus	⌚ Pendiente de sincronizar desde DigitalOcean
Producción DigitalOcean	⚠ Documentada desde Git; no validada en esta reconciliación
Backups	⌚ Pendiente
DNS local	⌚ Pendiente

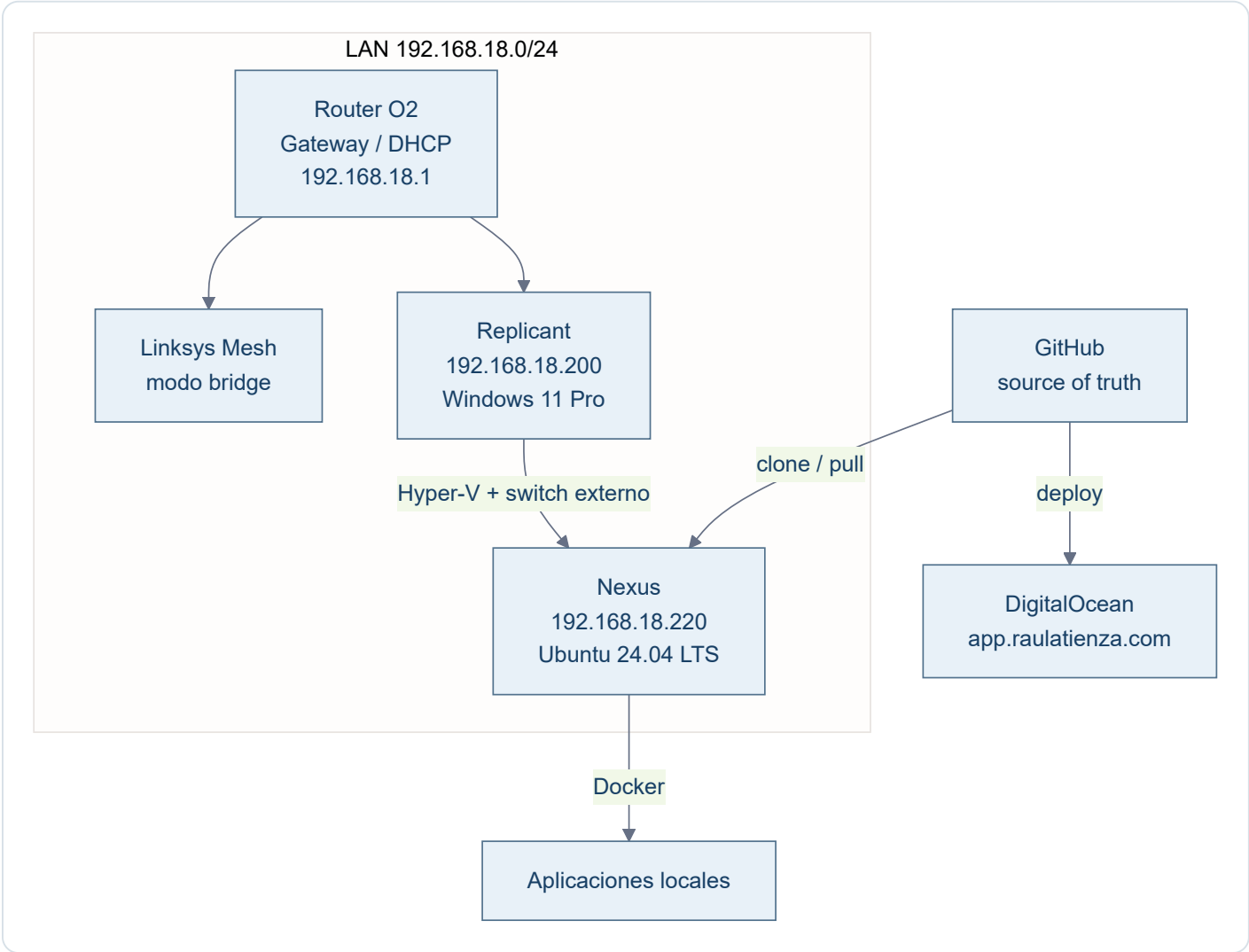
Cómo usar esta documentación

- **Arquitectura** explica cómo encajan las piezas.
- **Fases** conserva el recorrido y las decisiones que llevaron al estado actual.
- **Hosts** describe cada máquina.
- **Red** documenta direccionamiento e inventario.
- **Despliegue** fija los patrones Git/Docker.
- **Aplicaciones** contiene solo la ficha de infraestructura de cada app.
- **Operación** concentra comandos y procedimientos cortos.
- **Pendientes** conserva el estado y los encargos que deben retomarse sin depender de memoria de chat.
- **Decisiones** registra criterios que no conviene redescubrir cada vez.

Los estados distinguen entre contenido **implementado en Git**, **probado localmente**, **validado u observado en Nexus** y **validado en producción**. Una evidencia de un entorno no se extrapola a otro.

Arquitectura

Modelo general



Reparto de responsabilidades

Elemento	Responsabilidad
Replicant	Puesto de trabajo, UI, Hyper-V y administración del lab
Nexus	Ejecución local de servicios Linux y contenedores
GitHub	Código, configuración versionable e histórico
DigitalOcean	Servicios públicos o que necesiten disponibilidad 24x7
/opt/data	Persistencia local
/opt/secrets	Secretos y configuración privada fuera de Git

Elemento	Responsabilidad
/opt/backups	Copias; política aún pendiente

Regla arquitectónica

Regla base

Si una necesidad puede resolverse como contenedor sin ensuciar el host, se prefiere Docker. El host Ubuntu se mantiene deliberadamente pequeño.

Qué no se instala por defecto

- DNS local.
- Reverse proxy.
- Portainer, Cockpit o Webmin.
- Fail2ban mientras SSH permanezca restringido a la LAN.
- Suites de monitorización complejas.
- Automatización de backups hasta definir política.

Fase 01 · Concepto

Objetivo

Construir un laboratorio doméstico sencillo para desarrollar y ejecutar aplicaciones sin mezclar estación de trabajo, servidor y cloud.

Decisiones

- **Replicant** permanece como equipo Windows de trabajo.
- Se crea una VM Linux dedicada: **Nexus**.
- GitHub será la fuente de verdad para código/configuración.
- Docker será la capa de ejecución estándar en Linux.
- DigitalOcean se reserva para servicios públicos/24x7.
- Los datos sensibles pueden permanecer solo en local.

Resultado

Se define una arquitectura híbrida con separación clara entre **desarrollo**, **ejecución local**, **control de versiones** y **cloud**.

Fase 02 · Replicant

Objetivo

Preparar el host Windows que aloja Hyper-V y sirve de estación de trabajo principal.

Configuración consolidada

- Windows 11 Pro.
- 16 GB RAM.
- Intel N100, 4 cores.
- Hyper-V habilitado.
- Switch externo: Replicant Ethernet .
- IP fija: 192.168.18.200 .
- Gateway: 192.168.18.1 .

Rol

Replicant concentra las herramientas interactivas y administra los servidores, pero evita asumir cargas de servidor que encajan mejor en Nexus.

Fase 03 · Nexus

Objetivo

Crear un servidor Linux local, estable y mínimo.

VM

- Hyper-V Gen2.
- Ubuntu Server 24.04.4 LTS.
- Aproximadamente 4 vCPU.
- Aproximadamente 6 GB de RAM dinámica.
- VHDX dinámico de 150 GB.

Red

- Interfaz: `eth0` .
- MAC virtual: `00:15:5d:12:7b:00` .
- IP fija: `192.168.18.220/24` .
- Gateway: `192.168.18.1` .
- Netplan: `/etc/netplan/99-nexus-static.yaml` con permisos `600` .

Acceso

SSH por clave desde Replicant mediante el alias `nexus` .

Resultado

Nexus queda independiente del DHCP para su dirección y preparado para convertirse en host de servicios.

Fase 04 · Docker y Git

Docker

Se instala Docker CE desde el repositorio oficial y Docker Compose. El usuario `raul` pertenece al grupo `docker` y puede operar sin `sudo` para comandos Docker.

Git

Git queda configurado en Nexus con identidad propia. Se genera una clave SSH específica **Nexus** → **GitHub** y se valida el acceso.

Estructura `/opt`

```
/opt/  
├─ apps/  
├─ data/  
├─ backups/  
├─ compose/  
└─ secrets/
```

Seguridad básica

- UFW activo.
- Política: `deny incoming`, `allow outgoing`.
- SSH permitido solo desde `192.168.18.0/24`.
- `unattended-upgrades` activo y habilitado.

Resultado

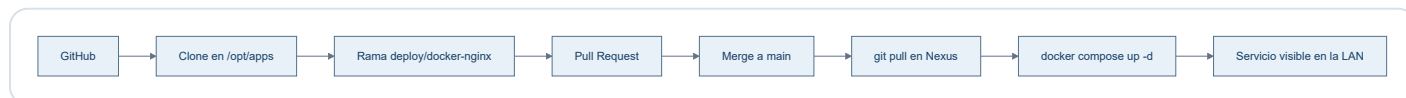
Nexus queda listo como host de aplicaciones reproducibles y versionadas.

Fase 05 · PoC Salones AV

Objetivo

Validar el patrón completo con una aplicación real y simple.

Flujo probado



Resultado

- Repo: `ratienza/salones-av-valencia-palace`.
- Ruta: `/opt/apps/salones-av-valencia-palace`.
- Imagen: `nginx:alpine`.
- Contenedor: `salones-av`.
- Puerto: `192.168.18.220:8081 → 80/tcp`.
- HTML montado en solo lectura desde `proyecto_html`.

El PoC confirma el patrón **GitHub** → **Nexus** → **Docker** → **LAN**.

Host · Replicant

Campo	Valor
Rol	Estación de trabajo + host Hyper-V
SO	Windows 11 Pro
IP	192.168.18.200
Gateway	192.168.18.1
Virtualización	Hyper-V
Switch	Replicant Ethernet

Responsabilidades

- Herramientas interactivas: ChatGPT, Codex, Gemini y similares.
- Administración de Nexus.
- Hyper-V.
- Punto de entrada SSH hacia Nexus y DigitalOcean.

Criterio

No convertir Replicant en servidor por comodidad si el servicio puede vivir limpiamente en Nexus.

Host · Nexus

Campo	Valor
Hostname	nexus
SO	Ubuntu Server 24.04.4 LTS
IP	192.168.18.220/24
Gateway	192.168.18.1
Interfaz	eth0
MAC	00:15:5d:12:7b:00
Plataforma	VM Hyper-V Gen2

Software base

- OpenSSH Server.
- Docker Engine CE.
- Docker Compose.
- Git.
- UFW.
- Nano.
- unattended-upgrades.

Estructura

/opt/apps	repositorios y aplicaciones
/opt/data	datos persistentes
/opt/backups	copias
/opt/compose	composiciones separadas si hacen falta
/opt/secrets	secretos y .env fuera de Git

Seguridad

Estado
SSH por clave, UFW activo y actualizaciones automáticas de seguridad habilitadas.

Puertos conocidos

Puerto	Uso	Ámbito
22/tcp	SSH	LAN

Puerto	Uso	Ámbito
8081/tcp	Salones AV	LAN
8082/tcp	Replicant Lab · Nginx estático	LAN
8083/tcp	Reserva-Pistas-UTP · Nginx	LAN
53	systemd-resolved	localhost

Estado observado el 09/08/2026

- Docker y `unattended-upgrades` activos.
- Replicant Lab ejecutándose como sitio estático en Nginx, construido desde el `Dockerfile` y publicado en `192.168.18.220:8082 → 80`, sin bind mounts ni servidor de desarrollo.
- Salones AV accesible mediante Nginx en `192.168.18.220:8081`.
- Reserva-Pistas-UTP ejecutándose como backend privado y proxy Nginx en `192.168.18.220:8083`.

Esta observación confirma el estado del laboratorio privado en esa fecha; no valida DigitalOcean ni sustituye las definiciones versionadas de cada repositorio.

Runtime documental validado en Nexus

El PR #9 fusionado sustituye `mkdocs serve` y los bind mounts por una imagen construida desde el `Dockerfile`: MkDocs estricto en la etapa builder y Nginx estático en runtime, manteniendo `192.168.18.220:8082`. El 09/08/2026 se reconstruyó y recreó exclusivamente este servicio en Nexus y se validaron HTTP, navegación, recursos, cinco diagramas Mermaid y descargas HTML/PDF idénticas byte a byte a los artefactos versionados. El HTML funciona offline sin dependencias esenciales externas y el PDF conserva la documentación completa.

Host · DigitalOcean

Campo	Valor
Alias SSH	docean
Host	app.raulatienza.com
Usuario actual	root
Rol	Servicios públicos / 24x7

Patrón esperado

DigitalOcean debe seguir el mismo principio que Nexus: **despliegue desde Git**, separación de secretos y mínima configuración manual irreproducible.

Límite importante

Los datos sensibles no se sincronizan automáticamente entre Nexus y DigitalOcean. Solo se trasladan cuando exista una decisión explícita.

Nivel de validación

La configuración de producción descrita en este proyecto procede de la documentación versionada del repositorio de Reserva-Pistas-UTP. DigitalOcean no se inspeccionó durante la reconciliación documental del 09/08/2026, por lo que no se afirma una validación viva de su servicio, datos, procesos o configuración.

Host lógico · GitHub

GitHub no es un host de ejecución del lab, pero sí una pieza de infraestructura crítica porque actúa como **fuentes de verdad**.

Reglas

- Código y configuración versionable viven aquí.
- Cambios relevantes se realizan en rama.
- Se revisan mediante Pull Request.
- `main` representa el estado estable aprobado.
- Datos, bases de datos, `.env`, tokens y claves privadas no se versionan.

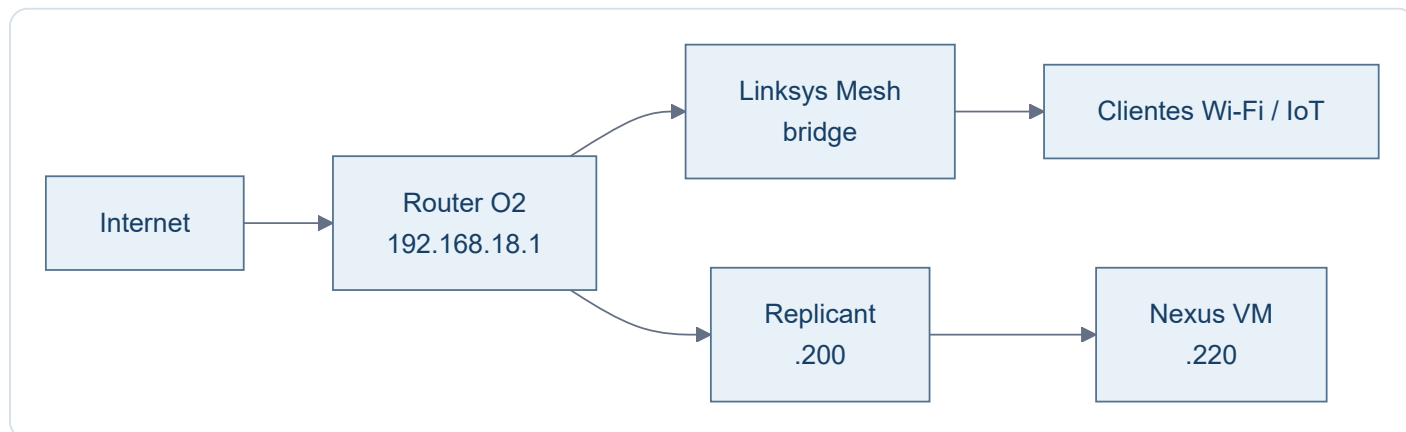
Repos relevantes

Repo	Función
ratienza/infra-replicant-lab	Documentación viva de infraestructura
ratienza/salones-av-valencia-palace	Aplicación AV / PoC Docker
ratienza/cartera-estrategica	Aplicación de cartera
ratienza/Reserva-Pistas-UTP	Aplicación de reservas de pádel

Cada aplicación conserva en su propio repositorio su código, configuración reproducible y documentación funcional. Este repositorio solo mantiene la visión de infraestructura y las diferencias comprobadas entre entornos.

Red · Visión general

Topología



Convención provisional

Rango	Uso previsto
.1	Router / gateway
.2 - .19	Infraestructura de red / mesh
.20 - .179	Clientes, IoT y DHCP general
.180 - .199	Equipos fijos, NAS, impresoras
.200 - .219	Servidores físicos
.220 - .239	VMs / laboratorio
.240 - .254	Reserva futura

DNS

Se deja pendiente. Para dos equipos de trabajo, la operación por IP es aceptable y evita añadir una dependencia innecesaria por ahora.

Red · Inventario provisional

Estado

Inventario de trabajo obtenido del escaneo de agosto de 2026. No se considera una CMDB definitiva y algunos nombres/fabricantes pueden requerir validación posterior.

Infraestructura y equipos conocidos

IP	Nombre / función	MAC / fabricante / nota
.1	Router Fibra O2	2C-96-82-69-25-F2 · gateway
.3	RAN-CASA	HP
.4	Sonos 5	5C-AA-FD-0D-54-36 · no visto
.5	Enchufe Emma	4C-11-AE-14-10-6E · IoT
.6	Keres.local	14-91-82-8D-8C-6A · Linksys
.7	SONOS Cocina	B8-E9-37-E7-51-2E
.8	Enchufe ordenador RAN-CASA	D4-A6-51-E5-0F-24 · Tuya
.9	Termostato Nest	18-B4-30-C4-52-48
.10	Sonos Habitación	B8-E9-37-E1-8E-AE
.11	Echo Darío	F4-03-2A-7B-1B-1B
.12	Alexa Comedor	DC-91-BF-D1-35-B3
.13	Linksys13497.local	14-91-82-8D-8C-B2 · Linksys
.14	Alexa Despacho	20-A1-71-00-43-44
.18	Play Darío	0C-FE-45-37-AA-4F · Sony
.19	Alexa Cocina	1C-FE-2B-4F-5A-1B
.20	Fire Stick	34-AF-B3-DE-58-06
.25	IoT desconocido	28-6D-CD-49-AF-28 · no visto
.27	Lamparita Darío	28-6D-CD-56-C8-41
.35	Lamparita comedor	28-6D-CD-5D-BD-81
.42	Chromecast antiguo	8A-05-36-D5-F8-2D · no visto
.44	Escalera	40-F5-20-C1-C7-FB · luces · no visto
.45	Lamparita comedor escalera	28-6D-CD-49-AB-D1

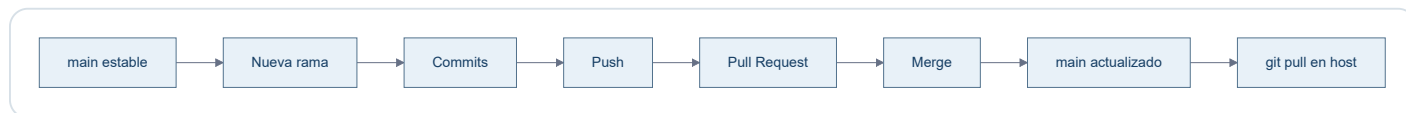
IP	Nombre / función	MAC / fabricante / nota
.52	Enchufe Raúl	D4-A6-51-E6-94-8B
.53	Sebastian - CECOTEC	44-87-63-97-02-EA
.54	Impresora Darío	28-C5-C8-AB-D7-BE · HP
.71	Luz despacho	CE-CC-5A-B6-E9-BC · no visto
.74	Móvil Emma	76-9D-67-FE-0D-CA · Samsung · no visto
.82	Chromecast	F6-45-CE-C2-7C-F6
.83	Móvil Darío	BA-D8-D1-F2-04-44 · no visto
.94	Portátil SH	28-92-00-45-01-CD · HP · no visto
.101	Móvil Emma antiguo	58-02-05-B8-4E-BA · no visto
.106	Móvil Raúl	52-1E-45-1D-FB-AA
.123	Antigua IP DHCP de Replicant	00-E0-4C-4B-35-AD · migrado a .200
.124	No identificado	A4-E8-8D-08-12-44 · no visto
.125	Antigua IP DHCP de Nexus	00-15-5D-12-7B-00 · migrado a .220
.200	Replicant	host físico Windows / Hyper-V · IP fija
.220	Nexus	VM Ubuntu / Docker · IP fija

Nota operativa

El inventario sirve para orientación y futuras decisiones de direccionamiento. No se reubicarán dispositivos por estética ni se convertirán en IP fija salvo necesidad concreta.

Despliegue · Git

Flujo normal



Convención de ramas para esta documentación

Las ramas representan **cambios lógicos**, no archivos individuales.

Ejemplos:

- docs/bootstrap-infra
- docs/app-salones
- docs/app-cartera
- docs/network
- docs/backups

Regla

Main

`main` debe representar siempre la documentación aprobada y coherente con el estado real.

Despliegue · Docker

Patrón

```
GitHub
↓
/opt/apps/<repo>
↓
compose.yml
↓
docker compose up -d
↓
servicios
```

Buenas prácticas del lab

- Preferir imágenes oficiales cuando sea razonable.
- Minimizar paquetes instalados en el host.
- Persistencia fuera del contenedor.
- Secretos fuera de Git.
- Publicar solo puertos necesarios.
- Cuando proceda, ligar puertos a `192.168.18.220` en vez de `0.0.0.0`.
- Separar proxy, backend y datos cuando cada pieza tenga una responsabilidad clara.
- Ejecutar los procesos de aplicación con un usuario no root siempre que sea viable.
- Usar `restart: unless-stopped` para servicios que deben recuperarse tras reiniciar Nexus.
- No añadir paneles de administración si no resuelven un problema real.

Convención de puertos en Nexus

Puerto	Servicio	Estado
8080	Launch-pad de Nexus	Reservado
8081	Salones AV	Operativo
8082	Replicant Lab · documentación	Operativo
8083	Reserva-Pistas-UTP	Operativo
8084+	Próximos servicios	Asignación secuencial

Reglas:

- `8080` queda reservado como puerta de entrada del laboratorio y futuro launch-pad.
- A partir de `8081`, los servicios se numeran consecutivamente salvo necesidad técnica justificada.
- Cada nuevo servicio debe quedar documentado aquí al asignar su puerto.
- Siempre que sea posible, publicar el servicio ligado a `192.168.18.220` y no a `0.0.0.0`.

Reserva-Pistas-UTP · patrón validado

Reserva-Pistas utiliza un Compose de dos servicios:

```

LAN :8083
↓
nginx
↓
red_privada Compose
↓
app:8765

```

El backend no publica 8765 hacia el host. Nginx lo alcanza mediante el DNS interno de Docker usando el nombre de servicio `app`.

Los datos se desacoplan del ciclo de vida del contenedor mediante:

```
/opt/data/reserva-pistas:/app/data
```

La aplicación recibe por entorno:

```

APP_HOST=0.0.0.0
APP_PORT=8765
APP_DATA_DIR=/app/data

```

El código conserva valores por defecto compatibles con despliegues no Docker.

Operación estándar

```

cd /opt/apps/<proyecto>
git switch main
git pull
docker compose up -d --build

```

Comprobaciones habituales:

```

docker compose ps
docker compose logs --tail 50

```

Parada y recreación:

```

docker compose down
docker compose up -d

```

Un `down` elimina contenedores y la red del proyecto, pero no debe eliminar datos persistentes alojados fuera del contenedor.

Autoarranque

Docker Engine arranca con Ubuntu. Los contenedores existentes con `restart: unless-stopped` se recuperan automáticamente después de reiniciar Nexus.

Compose no necesita ejecutarse manualmente durante el arranque para recuperar esos contenedores ya creados; su fichero YAML sigue siendo la definición reproducible para recrearlos o actualizarlos.

Replicant Lab · sitio estático reproducible

La definición versionada converge en un único modelo:

```
mkdocs.yml + docs/
  ↓
Dockerfile · MkDocs build --strict
  ↓
sitio estático
  ↓
Nginx :80
  ↓
192.168.18.220:8082
```

`compose.yml` construye el `Dockerfile` , publica únicamente `192.168.18.220:8082 → 80` y no monta fuentes ni ejecuta `mkdocs serve` .

Actualización desplegada

```
cd /opt/apps/infra-replicant-lab
git switch main
git pull --ff-only origin main
docker compose up -d --build
```

Cada cambio documental desplegado requiere reconstruir la imagen porque Nginx sirve la copia estática incluida durante el build. Las dependencias viven en las etapas versionadas; no se instalan en Nexus.

Separación de estados

El modelo estático está implementado, probado localmente y validado en Nexus. La validación del 09/08/2026 incluyó el contenedor Nginx estable, HTTP y navegación, cinco diagramas Mermaid, descargas reales idénticas a Git, HTML offline y PDF completo. No implica validación en producción.

Operación · SSH

Desde Replicant

```
ssh nexus  
ssh docean
```

Configuración conceptual

```
Host nexus  
  HostName 192.168.18.220  
  User raul  
  IdentityFile C:\Users\raul\.ssh\nexus_local  
  IdentitiesOnly yes  
  
Host docean  
  HostName app.raulatienza.com  
  User root  
  IdentityFile C:\Users\raul\.ssh\reserva_pistas_do  
  IdentitiesOnly yes
```

Claves

- `nexus_local` : Replicant → Nexus.
- `reserva_pistas_do` : Replicant → DigitalOcean.
- `~/.ssh/id_ed25519` en Nexus: Nexus → GitHub.

Nunca documentar

No almacenar en este repo claves privadas, tokens, contraseñas ni el contenido de secretos.

Operación · Comandos útiles

Host

```
ip addr  
ip route  
sudo ss -tulpn
```

Firewall

```
sudo ufw status verbose
```

Docker

```
docker ps  
docker compose up -d  
docker compose down  
docker compose logs
```

Git

```
git status  
git switch main  
git pull
```

Actualizaciones

```
sudo apt update  
apt list --upgradable  
sudo apt upgrade
```

Filosofía

Este documento evita convertirse en un recetario infinito. Solo se incluyen comandos frecuentes y útiles para operar el lab.

Operación · Mantenimiento

Actualizaciones

`unattended-upgrades` está activo y habilitado. Ubuntu puede diferir ciertos paquetes mediante phased rollout; no se fuerzan salvo necesidad.

Comprobación:

```
systemctl status unattended-upgrades --no-pager
```

Limpieza

`apt autoremove` puede utilizarse tras revisar qué paquetes se eliminarán.

Criterio de mantenimiento

- Actualizar con regularidad.
- No instalar herramientas "por si acaso".
- Revisar servicios expuestos con `ss -tulpn`.
- Mantener `main` coherente con la realidad.

Descargas

La documentación MkDocs de este proyecto es la única referencia canónica. Los dos ficheros siguientes se generan exclusivamente desde `mkdocs.yml`, las páginas de `docs/` incluidas en `nav` y sus recursos versionados.

[↓ Descargar documentación HTML offline](#)[↓ Descargar documentación PDF](#)

Rutas canónicas únicas

Artefacto	Ruta	Cobertura
HTML autocontenido	<code>docs/downloads/Replicant-Lab.html</code>	Toda la navegación MkDocs, índice interno y cinco Mermaid
PDF A4	<code>docs/downloads/Replicant-Lab.pdf</code>	Portada, índice, documentación completa, numeración y cinco Mermaid

No se mantiene ninguna copia alternativa.

Propiedades verificables

- Ambos artefactos muestran la misma huella SHA-256 de las fuentes.
- El HTML incorpora CSS, JavaScript y Mermaid localmente y funciona mediante `file://` sin red.
- El HTML no solicita CDN ni contiene clientes de recarga, conexiones dinámicas o referencias al servidor de desarrollo.
- El PDF mantiene texto seleccionable, enlaces, tablas, código, avisos y diagramas.
- El JavaScript del sitio calcula las rutas desde su propio recurso y funciona con el sitio estático en `/`.

Regeneración y sincronía

Después de preparar las dependencias fijadas, un único comando regenera y valida todo:

```
python scripts/docs_pipeline.py generate
```

CI ejecuta el modo `check`, vuelve a renderizar un PDF de control, verifica la huella y cobertura, construye la imagen Docker final, arranca Nginx temporalmente y compara byte a byte las descargas servidas con los artefactos versionados.

Estado de despliegue

El pipeline y el runtime estático están implementados, probados localmente y validados en Nexus. El 09/08/2026 se comprobaron el sitio publicado, los cinco diagramas y las descargas reales; los ficheros servidos coincidieron byte a byte con los artefactos versionados. Esta validación corresponde al laboratorio privado y no se extrapola a producción.

Pendientes

Panel compacto del estado de los repositorios GitHub vinculados a Raul Lab. Corte comprobado: **09/08/2026**. Los detalles permanecen plegados para facilitar una lectura rápida y conservan los hitos, pendientes, siguientes acciones y PR ya documentados.

Leyenda

- Terminado u operativo
- En desarrollo o con pendientes
- Bloqueado o con incidencia real
- Sin iniciar

El texto acompaña siempre al indicador visual; el color no es la única señal de estado.

Estado conjunto

Repositorio	Finalidad	Estado	PR abiertos	Detalle
infra-replicant-lab	Gestor de infraestructura y operación de Raul Lab.	● Terminado y operativo en Nexus	0	Último hito: pipeline reproducible, MkDocs canónico, Nginx estático y portables completos. Terminado: validación real en Nexus y cierre documental mediante PR #11 . Pendientes: ninguno del gestor documental; DNS local y backups generales pertenecen a infraestructura. Siguiente acción: mantenimiento normal cuando cambie la fuente canónica. PR: último fusionado #11.
Reserva-Pistas-UTP	Automatización de reservas de pádel.	● Operativo con pendientes	1	Último hito: PR #3 , despliegue Docker Compose reproducible para Nexus. Terminado: servicio operativo en Nexus. Pendientes: promoción controlada y sincronización segura del estado vivo desde DigitalOcean; decidir el destino del PR borrador #1. Siguiente acción: actuar solo mediante encargo explícito de producción. PR: abierto #1 borrador ; último fusionado #3.
salones-av-valencia-palace	Documentación operativa audiovisual.	● Operativo con incidencia conocida	0	Último hito: PR #1 , Compose con Nginx. Terminado: HTML y recursos operativos versionados y servicio observado en Nexus. Pendientes: reconciliar en su repositorio el bind LAN observado y completar la revisión final indicada en su contexto. Siguiente acción: corregir la deriva en el repositorio de la aplicación. PR: ninguno abierto; último fusionado #1.
cartera-estrategica	MVP privado de análisis de cartera.	● En desarrollo	0	Último hito: PR #17 , estabilización del arranque privado. Terminado: fases 0–7B y libro de cash integrados: versión documentada v1.2.0. Pendientes: Fase 8, pruebas y endurecimiento; Fase 9, publicación privada. La Fase 7C permanece fuera del MVP. Siguiente acción: continuar con la Fase 8. PR: ninguno abierto; último fusionado #17.
CV-Raul-IA-Estudio-Google-	CV web generado desde Google AI Studio.	● En desarrollo	0	Último hito: Dockerfile y configuración Nginx; aplicación Vite y PDF generado. Terminado: aplicación desplegable versionada. Pendientes: ninguno documentado en README. Siguiente acción: no definida. PR: ninguno abierto; último cambio directo 38c9fe7.
Apps Launch	Launchpad público de aplicaciones.	● Operativo según README	0	Último hito: la tarjeta del CV apunta a Cloud Run. Terminado: despliegue y verificación documentados. Pendientes: ninguno versionado. Siguiente acción: incorporar o cambiar una aplicación cuando exista un encargo. PR: ninguno abierto; último cambio directo 98149fa.
Consumos Cupra	Control de consumos de combustible.	● En desarrollo	0	Último hito: soporte para despliegue bajo la ruta consumos. Terminado: PWA, servidor y Docker Compose versionados. Pendientes: ninguno documentado en README. Siguiente acción: no definida. PR: ninguno abierto; último cambio directo 05b535f.
control-red	Panel local de inventario y escaneo de red.	● Operativo local según README	0	Último hito: primera versión del panel PowerShell. Terminado: lanzador local e inventario persistente. Pendientes: ninguno documentado. Siguiente acción: proteger inventario y snapshots ante cualquier cambio. PR: ninguno abierto; último cambio directo 0e285d2.

Exclusión expresa

[ratienza/python](#) no forma parte de Raul Lab y se excluye de esta visión.

Reglas de mantenimiento

- Consultar GitHub antes de cambiar estados, PR o hitos.
- No convertir documentación de despliegue en validación viva.
- No presentar como pendiente lo ya cerrado.
- Registrar “sin pendiente documentado” cuando GitHub y el repositorio no definan una siguiente acción.
- Mantener separados código, secretos, bases, históricos y datos vivos de cada aplicación.

Aplicaciones

Esta sección contiene **solo la ficha de infraestructura** de cada aplicación.

La documentación funcional, de negocio o de desarrollo detallada permanece en el repositorio propio de cada app.

Aplicación	Repo	Host	Estado
Salones AV	<code>ratienza/salones-av-valencia-palace</code>	Nexus	✅ Desplegada
Reserva-Pistas-UTP	<code>ratienza/Reserva-Pistas-UTP</code>	Nexus / DigitalOcean	✅ Nexus validado · producción documentada en DigitalOcean, no verificada aquí
Cartera Estratégica	<code>ratienza/cartera-estrategica</code>	Replicant / Windows local	✅ MVP Streamlit v1.2.0 · Nexus no definido

Aplicación · Salones AV

Ficha de infraestructura

Campo	Valor
Repo	ratienza/salones-av-valencia-palace
Host	Nexus
Ruta	/opt/apps/salones-av-valencia-palace
Contenedor	salones-av
Imagen	nginx:alpine
Puerto	192.168.18.220:8081 → 80/tcp
Contenido	proyecto_html montado en solo lectura

Comportamiento de despliegue

El contenido HTML se sirve mediante un bind mount. Por tanto, un `git pull` actualiza los ficheros visibles sin necesidad de reconstruir una imagen.

Diferencia observada entre Git y Nexus

El `compose.yml` vigente en `main` publica `8081:80`, mientras el checkout desplegado en Nexus contiene un cambio local no versionado que lo restringe a `192.168.18.220:8081:80`. El contenedor observado usa efectivamente el bind a la IP de Nexus.

La restricción local mejora el alcance de red, pero constituye deriva respecto a GitHub. Debe reconciliarse en el repositorio propio de Salones AV antes de considerar el despliegue plenamente reproducible; esta documentación no corrige ni adopta silenciosamente ese cambio.

Alcance de esta ficha

La documentación técnica/funcional específica de los salones permanece en el repo de la aplicación.

Reserva-Pistas-UTP

Ficha de infraestructura de la aplicación de reservas de pádel de Torre de Porta-Coeli.

La documentación funcional y de negocio permanece en el repositorio de la aplicación: `ratienza/Reserva-Pistas-UTP` . Esta ficha describe exclusivamente cómo se despliega y opera dentro de Replicant Lab y cómo se diferencia de producción.

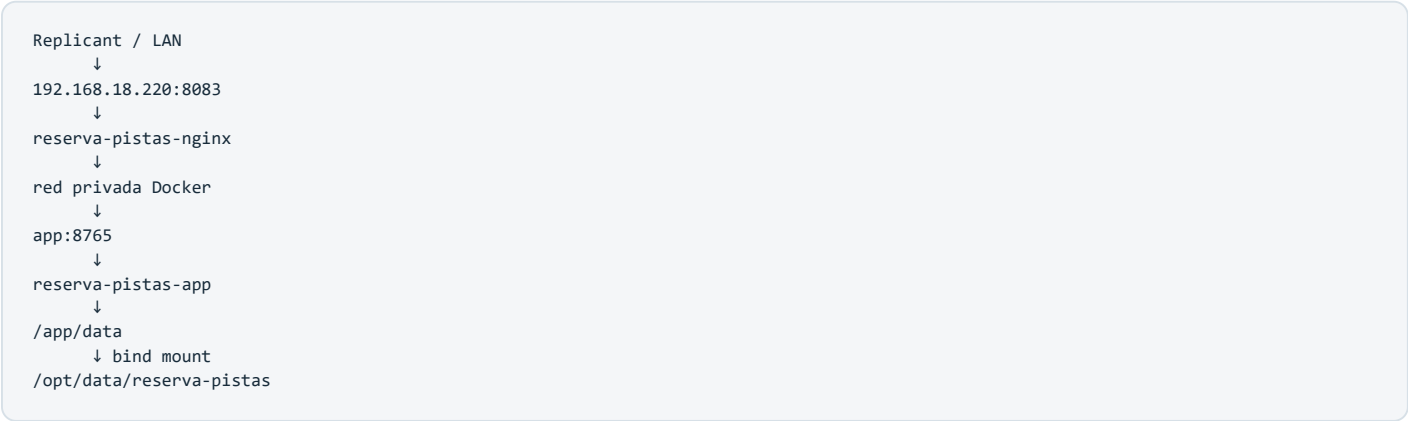
Estado

Elemento	Valor
Repositorio	<code>ratienza/Reserva-Pistas-UTP</code>
Nexus	✅ Operativo en Docker Compose
Ruta Nexus	<code>/opt/apps/Reserva-Pistas-UTP</code>
URL LAN	<code>http://192.168.18.220:8083</code>
Backend interno Docker	<code>app:8765</code>
Datos persistentes Nexus	<code>/opt/data/reserva-pistas</code>
Producción	DigitalOcean
URL producción	<code>https://app.raulatienza.com/padel/</code>
Producción modificada durante la adaptación	No

El 09/08/2026 se observó en Nexus que ambos contenedores estaban activos, el backend no publicaba puerto al host y Nginx exponía `192.168.18.220:8083` . El checkout de Nexus coincidía con `main` del repositorio de la aplicación. DigitalOcean no se inspeccionó durante esa comprobación.

Arquitectura Nexus

La versión de Nexus se ejecuta completamente en Docker Compose con dos servicios:



Solo Nginx publica un puerto hacia la LAN. El backend Python no se publica directamente.

Servicio `app`

- Imagen propia construida desde `Dockerfile` sobre `python:3.12-slim`.
- Ejecuta `python app.py`.
- Escucha en `0.0.0.0:8765` dentro del contenedor mediante variables de entorno.
- Ejecuta con UID/GID `1000:1000`, evitando crear datos persistentes como `root`.
- Usa `restart: unless-stopped`.

Servicio `nginx`

- Imagen oficial `nginx:alpine`.
- Publica `192.168.18.220:8083 -> 80/tcp`.
- Resuelve el backend mediante el nombre de servicio Docker `app:8765`.
- Usa `restart: unless-stopped`.

Qué aprendimos del montaje

La primera prueba se hizo con Python ejecutado manualmente en Nexus y un Nginx temporal con `--network host`. Sirvió para validar compatibilidad Linux sin tocar producción, pero no era un despliegue autónomo.

El montaje definitivo elimina esa dependencia manual:

1. `Dockerfile` empaqueta el backend Python.
2. `compose.yml` define backend y proxy como una sola aplicación desplegable.
3. Compose crea una red privada y registra los nombres de servicio.
4. Nginx puede resolver `app` sin conocer una IP fija del contenedor.
5. Solo Nginx publica `8083` en la LAN.
6. El estado vive fuera de los contenedores.
7. Docker reinicia ambos servicios automáticamente después de reiniciar Nexus.

Configuración por entorno

`app.py` conserva compatibilidad con el despliegue previo usando valores por defecto y permite cambiar el comportamiento mediante variables de entorno:

```
APP_HOST
APP_PORT
APP_DATA_DIR
```

Valores por defecto:

```
APP_HOST=127.0.0.1
APP_PORT=8765
APP_DATA_DIR=.
```

En Nexus, Compose define:

```
APP_HOST=0.0.0.0
APP_PORT=8765
APP_DATA_DIR=/app/data
```

Esto permite que el mismo código sirva para distintos entornos sin mantener dos variantes de `app.py`.

Persistencia

Los datos locales no forman parte de la imagen ni del repositorio Git.

En Nexus:

```
/opt/data/reserva-pistas
↓
/app/data
```

El bind mount contiene los ficheros de estado/configuración local que la aplicación utiliza:

```
credentials.local.json
notifications.local.json
tasks.local.json
telegram.offset.local
telegram.state.local
```

La persistencia se validó recreando contenedores y comprobando que los datos permanecían disponibles.

Seguridad y Git

`.gitignore` protege los ficheros locales de la aplicación y `.dockerignore` evita que `.git`, `.venv`, bytecode y ficheros `*.local*` entren en el contexto de construcción de Docker.

Los secretos, credenciales, estado y datos reales permanecen fuera de Git.

Firewall

UFW permite `8083/tcp` exclusivamente desde la LAN:

```
8083/tcp    ALLOW    192.168.18.0/24
```

Regla aplicada:

```
sudo ufw allow from 192.168.18.0/24 to any port 8083 proto tcp
```

Operación Nexus

Arranque / actualización

```
cd /opt/apps/Reserva-Pistas-UTP
git switch main
git pull
docker compose up -d --build
```

Estado

```
docker compose ps
```

Logs

```
docker logs reserva-pistas-app --tail 50
docker logs reserva-pistas-nginx --tail 50
```

Reinicio

```
docker compose restart
```

Parada completa

```
docker compose down
```

Los contenedores usan `restart: unless-stopped` ; tras un reinicio del host, Docker recupera automáticamente ambos servicios sin necesidad de ejecutar Compose manualmente.

Validaciones realizadas

- Imagen Python construida correctamente.
- Backend ejecutado dentro de Docker con UID/GID `1000:1000` .
- Comunicación `nginx -> app` mediante red privada de Compose.
- `GET /` responde HTTP `200` en `http://192.168.18.220:8083/` .
- Interfaz cargada correctamente desde Replicant.
- Bind mount `/opt/data/reserva-pistas:/app/data` validado.
- Persistencia conservada después de recrear contenedores.
- `restart: unless-stopped` validado.
- Reinicio completo de Nexus validado: la aplicación vuelve a quedar operativa automáticamente.

Diferencias Nexus / producción

Aspecto	Nexus / Lab	DigitalOcean / Producción
Objetivo	Desarrollo, prueba, staging y ejecución controlada	Servicio público 24x7
Backend	Contenedor <code>reserva-pistas-app</code>	Python gestionado por <code>systemd</code>
Proxy	Contenedor <code>reserva-pistas-nginx</code>	Nginx del host
Entrada	<code>192.168.18.220:8083</code>	<code>https://app.raulatienza.com/padel/</code>
Acceso	LAN	Internet + HTTPS + Basic Auth
Persistencia	<code>/opt/data/reserva-pistas</code>	Ficheros locales privados en <code>/opt/reserva-pistas/</code>
Git	Misma base de código	Misma base de código tras promoción controlada
Disponibilidad	Depende de Replicant/Nexus	24x7

Nexus puede ejecutar reservas reales porque usa el mismo código y puede disponer de credenciales válidas, pero no debe competir simultáneamente con producción sobre las mismas tareas.

Estado de `tasks.local.json`

La aplicación no utiliza una base SQL: el histórico, programaciones y estado operativo se conservan principalmente en `tasks.local.json`.

Antes de considerar Nexus plenamente sincronizado con producción, debe copiarse de forma controlada el `tasks.local.json` vigente de DigitalOcean.

Regla obligatoria:

1. DigitalOcean es la fuente del estado vivo actual hasta la sincronización.
2. Obtener el `tasks.local.json` más reciente de producción.
3. Validar integridad y conservar el histórico completo.
4. Revisar tareas con estado `queued` o `running`.
5. Si existen, neutralizarlas únicamente en la copia de Nexus antes de activar la ejecución allí.
6. No modificar producción durante esta operación.

Regla operativa crítica

No ejecutar simultáneamente tareas reales equivalentes en Nexus y DigitalOcean. Ambas instancias podrían intentar reservar la misma pista.

La promoción a producción y la sincronización autónoma del estado quedan dentro del encargo específico de Codex para esta aplicación.

Aplicación · Cartera Estratégica

Estado

Aplicación Python/Streamlit operativa como MVP local en Windows, con versión documental `v1.2.0` y SQLite privada fuera del repositorio. El repositorio de la aplicación registra las fases 0–7B integradas y mantiene pendientes el endurecimiento y una eventual publicación privada.

Entorno comprobado

El entorno vigente es Replicant/Windows con ejecución local de Streamlit. No existe en la documentación actual de la aplicación una decisión aprobada que convierta Nexus o Docker/Linux en su siguiente destino.

Consideraciones

- La base privada no debe entrar en Git.
- Tokens como EODHD deben inyectarse como secreto/entorno.
- Los datos de inversión pueden permanecer solo en local.
- Cualquier soporte Docker/Linux o despliegue remoto deberá decidirse e implementarse en el repositorio de la aplicación mediante rama y PR.

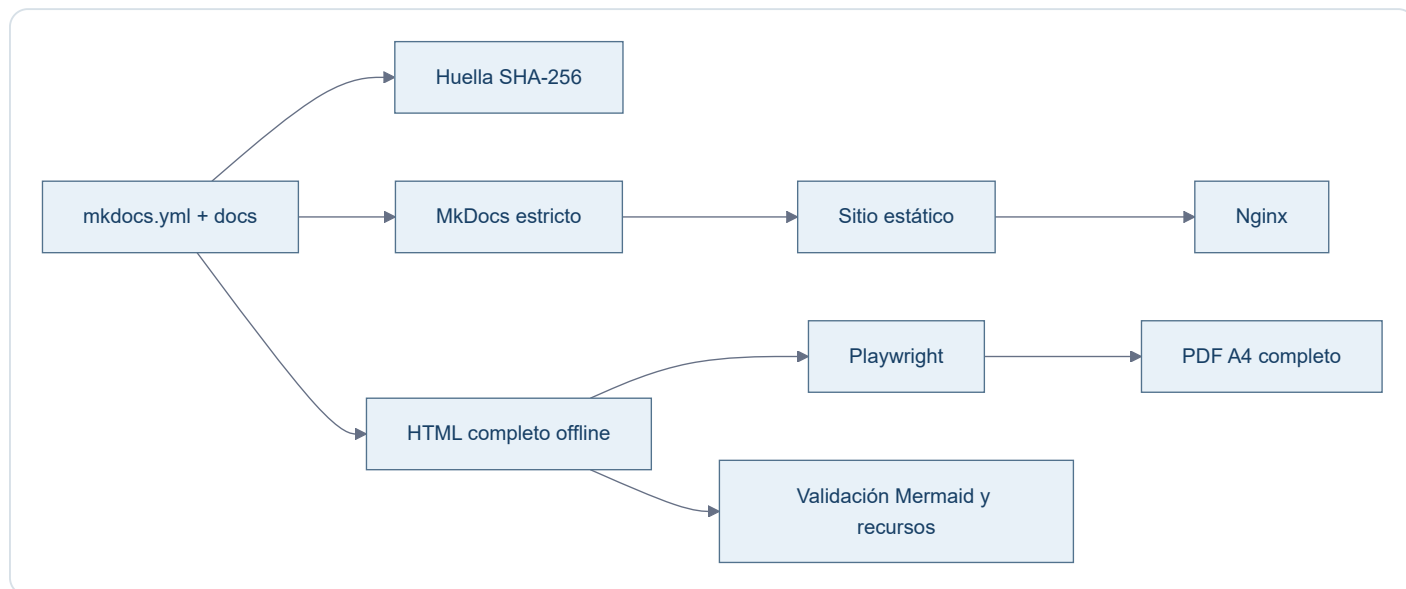
Pendiente

No está desplegada ni validada en Nexus. Tampoco debe presentarse Nexus como objetivo acordado hasta que exista una decisión versionada en el repositorio de la aplicación.

Autodocumentación

Estado implementado

La documentación mantiene una fuente humana canónica (`mkdocs.yml` , `docs/` y recursos) y una cadena reproducible para construir el sitio y sus artefactos derivados.



Herramientas versionadas

- `scripts/docs_pipeline.py` : orquesta construcción, generación, sincronía y validaciones estructurales.
- `scripts/render_portables.mjs` : renderiza Mermaid en Chromium y produce PDF y evidencias.
- `scripts/portable.css` : presentación compartida por HTML offline y PDF.
- `requirements-docs.txt` : dependencias Python exactas.
- `package.json` y `pnpm-lock.yaml` : Mermaid, Playwright y árbol Node fijados.
- `docs/javascripts/mermaid.min.js` : runtime derivado de la versión Mermaid fijada y comprobado contra `node_modules` .

Huella y reproducibilidad

La huella incluye configuración MkDocs, páginas en `nav` , recursos y herramientas que afectan a los portables. El HTML debe coincidir byte a byte con la salida esperada. El PDF se comprueba mediante esa huella visible, cobertura textual, número de páginas, enlaces y equivalencia de paginación con una regeneración de control.

Temporales

Todos los resultados intermedios viven bajo `.build/docs-pipeline/` . Entornos virtuales, `node_modules` , navegadores, cachés, capturas y reportes no entran en Git.

Flujo de actualización

1. modificar exclusivamente fuentes MkDocs;
2. ejecutar `python scripts/docs_pipeline.py generate ;`
3. revisar el sitio, HTML, PDF y capturas;
4. ejecutar `python scripts/docs_pipeline.py check ;`
5. revisar el diff completo;
6. commit, push y Pull Request;
7. tras merge, reconstruir el servicio estático en Nexus mediante Compose;
8. validar publicación y descargas en Nexus como un estado separado.

Decisiones arquitectónicas

Registro corto de decisiones que no conviene redescubrir.

Decisión	Motivo
Nexus usa IP fija <code>.220</code>	Identidad estable del servidor local
Replicant usa <code>.200</code>	Identidad estable del host físico
DNS local se pospone	Con dos equipos de trabajo, las IP son suficientes
Docker como estándar	Mantener limpio el host Ubuntu
GitHub como source of truth	Reproducibilidad e histórico
Datos y secretos fuera de Git	Seguridad y separación de responsabilidades
UFW restringe SSH a LAN	Reducir superficie de exposición
No usar Portainer/Webmin/Cockpit por defecto	Evitar complejidad innecesaria
Ramas por cambio lógico	Facilitar revisión y mantener <code>main</code> estable
MkDocs es la fuente documental canónica	<code>mkdocs.yml</code> , <code>docs/</code> y sus recursos definen contenido, estructura y presentación
HTML/PDF son artefactos derivados	Deben sincronizarse mediante un proceso reproducible; nunca prevalecen sobre MkDocs
Nexus sirve una imagen estática Nginx	<code>Dockerfile</code> , Compose y CI comparten build MkDocs estricto; el despliegue requiere reconstrucción
Validación por entorno	Git, prueba local, Nexus y producción son estados distintos y no se extrapolan
Dependencias documentales fijadas	Python/Node, MkDocs, Mermaid y Playwright se resuelven desde lockfiles versionados
Huella SHA-256 para portables	Evita timestamps variables y permite demostrar sincronía inequívoca con las fuentes
PDF tratado como binario	<code>.gitattributes</code> anula <code>diff=astextplain</code> y conversiones CRLF de Git para Windows
Rutas portables únicas	HTML y PDF viven exclusivamente en <code>docs/downloads/</code>

9 de agosto de 2026

Change Log diario de Raul Lab. Reúne los cambios documentales, técnicos y operativos de la fecha, del hito más reciente al más antiguo.

Corrección final de experiencia documental · PR #11

- El [PR #11](#) integra la corrección final de cronología, visión transversal de repositorios y experiencia portable.
- La sección **Cambios** pasa a presentarse como **Change Log**, con una cronología única, fechas visibles y contexto técnico y operativo.
- **Pendientes** amplía su alcance a los ocho repositorios vinculados a Raul Lab comprobados en GitHub; `ratienza/python` queda excluido expresamente.
- El HTML independiente recupera una experiencia multipágina dentro de un único fichero autocontenido: índice persistente, una vista documental activa, anterior/siguiente, fragmentos directos, historial atrás/adelante, búsqueda local y adaptación móvil.
- El PDF mantiene su modelo editorial completo; HTML y PDF continúan generándose desde la fuente MkDocs canónica y no desde exportaciones antiguas.

18:55 · Estado real de Nexus registrado · PR #10

- El [PR #10](#) integró la evidencia de despliegue y cerró los estados que todavía figuraban como pendientes.
- `main` quedó en `cd09dec4cb083f7a761e09648e23404cfd35da0c`.
- Nexus quedó sincronizado, con Nginx estático estable en `192.168.18.220:8082`, cinco Mermaid y descargas verificadas.

18:39 · Pipeline reproducible y runtime estático · PR #9

- El [PR #9](#) integró el pipeline reproducible de documentación.
- MkDocs — `mkdocs.yml`, `docs/` y sus recursos— quedó reafirmado como fuente canónica de contenido, estructura y presentación.
- Docker, Compose y CI convergieron en `mkdocs build --strict` y una imagen final Nginx; desaparecieron `mkdocs serve`, los bind mounts y el runtime de desarrollo en Nexus.
- Mermaid `11.16.1` pasó a servirse localmente, sin CDN, y los cinco diagramas quedaron disponibles también offline.
- Se generaron un HTML autocontenido completo y un PDF A4 completo desde las 27 páginas de navegación.
- Las rutas canónicas quedaron fijadas en `docs/downloads/Replicant-Lab.html` y `docs/downloads/Replicant-Lab.pdf`; se eliminó el HTML duplicado obsoleto.
- La validación local incluyó MkDocs estricto, enlaces, recursos, Docker, escritorio, móvil, HTML `file://`, PDF y ausencia de CDN, localhost, WebSocket y recarga en vivo.
- El squash resultante fue `f457c57b472515afe46025078c7342dd5a75a7f1`.

Despliegue y validación real en Nexus

- Se actualizó exclusivamente `/opt/apps/infra-replicant-lab` mediante avance rápido y `docker compose up -d --build`.
- El contenedor `infra-replicant-docs` quedó activo con Nginx `1.27.4`, reinicios `0` y publicación `192.168.18.220:8082 → 80`.
- Se comprobaron navegación, recursos, presentación de escritorio y móvil, cinco Mermaid, HTML offline y PDF completo.
- Las descargas reales coincidieron byte a byte con Git:
- HTML: `17a2746d2e11e26d0302a57633c5b1b05119348d8e4a4629f00d67ab8cfce6a1`.
- PDF: `028a135d7645912569c5d6c6c13b7c885b250084e3b269add7d10f89f627e13b`.
- Los logs finales no mostraron errores, `404`, recarga en vivo, WebSocket ni dependencias esenciales externas.

17:11 · Reconciliación y consolidación documental · PR #8

- El [PR #8](#) reconcilió las fuentes MkDocs con el estado comprobado de GitHub y Nexus.

- Se separaron con claridad Replicant/Hyper-V, Nexus, DigitalOcean y los repositorios propios de cada aplicación.
- README, navegación, operación, despliegue, aplicaciones y pendientes quedaron alineados; el procedimiento de Nexus distinguió primer arranque de actualización ordinaria.
- El squash resultante fue `9c31728bf0a4399dc736bea59f7ed50ed480bf3`.

16:07 · Contrato operativo raíz · PR #7

- El [PR #7](#) incorporó `AGENTS.md`.
- El contrato fijó la jerarquía entre verdad técnica versionada, documentación MkDocs canónica y artefactos HTML/PDF derivados.
- También formalizó separación de entornos, protección de datos, flujo rama–pruebas–PR–merge y prohibición de desplegar o modificar producción sin autorización.
- El squash resultante fue `01d859fac7b7907ae92cf65bda2ad50cfb0e738f`.

8 de agosto de 2026

Change Log diario de Raul Lab. Se conserva íntegramente el histórico previamente registrado para esta fecha.

- Replicant queda con IP fija `192.168.18.200`.
- Nexus queda con IP fija `192.168.18.220` mediante Netplan.
- SSH por clave consolidado.
- UFW activado con SSH permitido solo desde la LAN.
- Se verifican Docker, Git y actualizaciones automáticas.
- Salones AV queda accesible desde la LAN en `192.168.18.220:8081`.
- Replicant Lab queda publicado en Nexus por `192.168.18.220:8082`.
- Reserva-Pistas-UTP pasa de un staging manual a un despliegue Docker Compose completo en Nexus: backend Python + Nginx en red privada Docker, acceso LAN por `192.168.18.220:8083`.
- El backend de Reserva-Pistas se hace configurable mediante `APP_HOST`, `APP_PORT` y `APP_DATA_DIR`, manteniendo valores por defecto compatibles con el despliegue anterior.
- La persistencia de Reserva-Pistas se separa en `/opt/data/reserva-pistas` y se monta como `/app/data`.
- El backend Docker se ejecuta como UID/GID `1000:1000` y ambos servicios usan `restart: unless-stopped`.
- Se valida HTTP `200`, persistencia tras recreación de contenedores y recuperación automática tras reinicio completo de Nexus.
- UFW permite `8083/tcp` exclusivamente desde `192.168.18.0/24`.
- Se formaliza la configuración Nexus en `ratienza/Reserva-Pistas-UTP` y se integra en `main` mediante PR #3.
- DigitalOcean permanece sin cambios y continúa como producción de Reserva-Pistas-UTP.
- Queda pendiente sincronizar a Nexus el histórico/estado vigente de `tasks.local.json` desde DigitalOcean, verificando antes que no existan tareas activas que puedan duplicarse.
- La portada documental incorpora accesos de descarga al HTML autónomo y al PDF de Replicant Lab.
- DNS local y backups se mantienen pendientes conscientemente.
- Se consolida `infra-replicant-lab` como documentación viva.